```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LassoCV
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_regression
import matplotlib.pyplot as plt
import seaborn as sns

# Load Excel file
excel_file = '/content/drive/My Drive/Colab Notebooks/LA_Health_V2.xlsx'
df = pd.read_excel(excel_file)

# Initial Data Info
print("Columns:", df.columns.tolist())
print("Initial shape:", df.shape)

# 1. Remove Duplicates
df = df.drop_duplicates()
print("After dropping duplicates:", df.shape)

# 2. Impute Missing Values
# Numeric with mean
numeric_cols = ['Patient Age','Total Turnover','Total Cashflow','Total Outstanding']
for col in numeric_cols:
    if df[col].isnull().any():
        df[col].fillna(df[col].mean(), inplace=True)
# Categorical with mode
cat_cols = ['Service Centre','ICD10 Description','Gender']
for col in cat_cols:
    if df[col].isnull().any():
        df[col].fillna('Unknown', inplace=True)
#print("After imputing missing values:", df.shape)

# 3. Remove Outliers using IQR for numeric fields
def remove_outliers_IQR(data, columns):
    mask = pd.Series(True, index=data.index)
    for col in columns:
        Q1 = data[col].quantile(0.25)
        Q3 = data[col].quantile(0.75)
        IQR = Q3 - Q1
        low_bound = Q1 - 1.5*IQR
        high_bound = Q3 + 1.5*IQR
        mask &= (data[col] >= low_bound) & (data[col] <= high_bound)
    return data[mask]

numeric_for_outliers = ['Payment_Duration','Patient Age','Total Turnover',
                        'Total Cashflow','Total Outstanding',
                        'Total Medical Aid Outstanding','Total Patient Outstanding']
df = remove_outliers_IQR(df, numeric_for_outliers)
print("After outlier removal:", df.shape)
```

```
Columns: ['Payment_Duration', 'Service Centre', 'Option', 'ICD10 Description', 'Patient Age', 'Gender', 'Specialty', 'Debtor Status
Initial shape: (50000, 15)
After dropping duplicates: (34590, 15)
After outlier removal: (18967, 15)
<ipython-input-62-0a1fcb4d27c5>:32: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained as
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col


  df[col].fillna(df[col].mean(), inplace=True)
<ipython-input-62-0a1fcb4d27c5>:37: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained as
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col

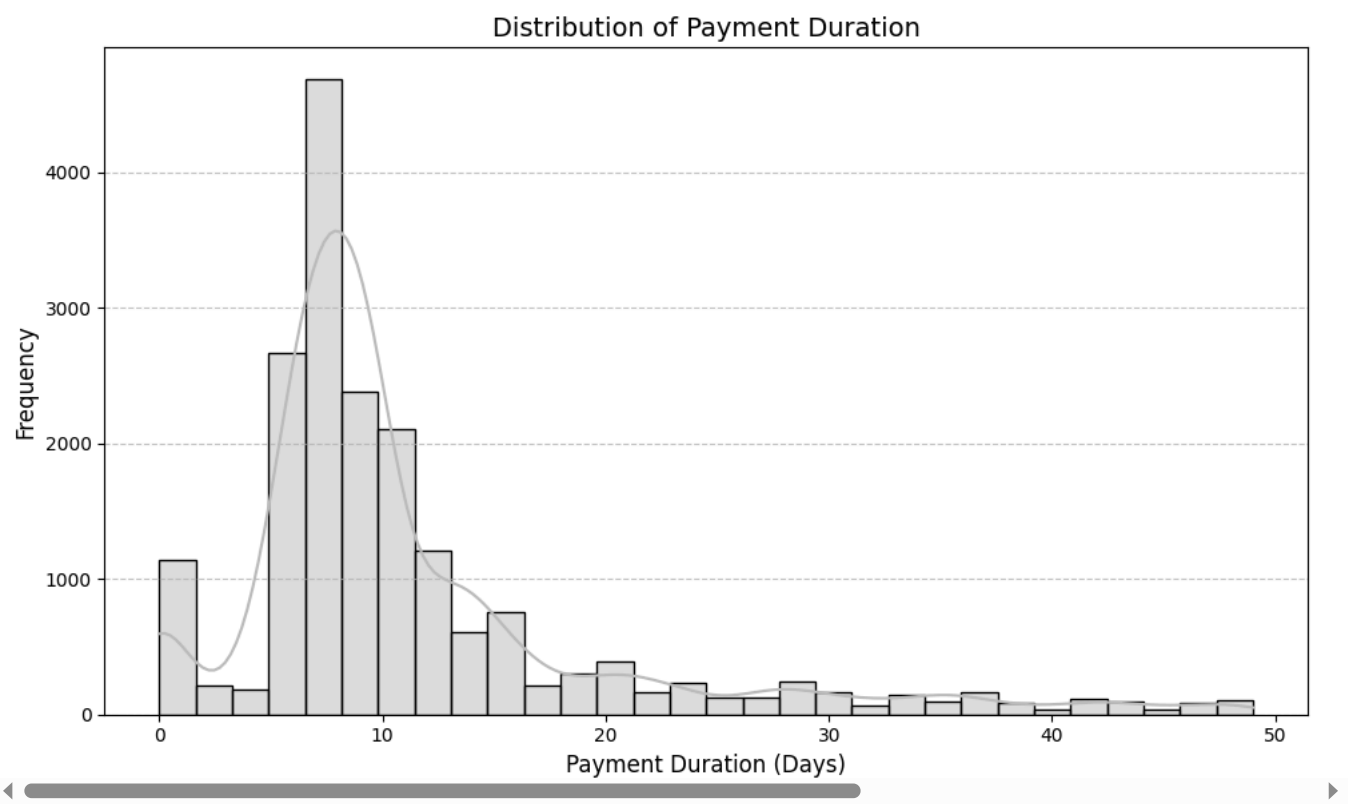

  df[col].fillna('Unknown', inplace=True)
```

```
desc_stats.to_excel("summary_statistics.xlsx")
desc_stats = desc_stats.round(2)
print(desc_stats)
```

```
                                  Min      Max     Mean       Q1  \
Payment_Duration                 0.00    49.00    11.40     7.00
Patient Age                     32.00    59.00    45.31    45.59
Total Turnover               -1349.66  4564.30  1143.82   430.60
Total Cashflow               -4413.20  1349.66 -1168.23 -1614.00
Total Outstanding                0.00     0.00     0.00     0.00
Total Medical Aid Outstanding    0.00     0.00     0.00     0.00
Total Patient Outstanding        0.00     0.00     0.00     0.00

                              Q2 (Median)       Q3  Std Dev
Payment_Duration                     9.00    13.00     8.92
Patient Age                         45.59    45.59     4.99
Total Turnover                     954.75  1538.10   913.76
Total Cashflow                    -968.00  -439.10   901.87
Total Outstanding                    0.00     0.00     0.00
Total Medical Aid Outstanding        0.00     0.00     0.00
Total Patient Outstanding            0.00     0.00     0.00
```

```
# Plot histogram
plt.figure(figsize=(10, 6))
sns.histplot(df['Payment_Duration'], bins=30, kde=True, color='skyblue')
plt.title('Distribution of Payment Duration', fontsize=14)
plt.xlabel('Payment Duration (Days)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```




```
# Table 2: Group by PMB status
table2 = df.groupby('POSSIBLE PMB CALCULATED').agg(
    Mean_Payment_Duration=('Payment_Duration', 'mean'),
    Median_Payment_Duration=('Payment_Duration', 'median'),
    Number_of_Claims=('Payment_Duration', 'count')
).reset_index()

# Rename PMB field for clarity if needed
table2.rename(columns={'POSSIBLE PMB CALCULATED': 'PMB Status'}, inplace=True)
print("Table 2: Average Payment Duration by PMB Status")
print(table2)

# Table 3: Group by Contract Status
table3 = df.groupby('CONTRACTED/NON-CONTRACTED').agg(
    Mean_Payment_Duration=('Payment_Duration', 'mean'),
```

```
    Median_Payment_Duration=('Payment_Duration', 'median'),
    Number_of_Claims=('Payment_Duration', 'count')
).reset_index()

# Rename Contract Status field for clarity
table3.rename(columns={'CONTRACTED/NON-CONTRACTED': 'Contract Status'}, inplace=True)
print("\nTable 3: Average Payment Duration by Contract Status")
print(table3)
```

```
Table 2: Average Payment Duration by PMB Status
  PMB Status  Mean_Payment_Duration  Median_Payment_Duration  Number_of_Claims
0    NON-PMB              10.614221                      8.0              5555
1        PMB              11.731509                      9.0             13412

Table 3: Average Payment Duration by Contract Status
  Contract Status  Mean_Payment_Duration  Median_Payment_Duration  \
0      CONTRACTED              10.816766                      9.0
1  NON-CONTRACTED              15.223013                     10.0

   Number_of_Claims
0             16438
1              2529
```

```
# Group by Service Centre and calculate mean Payment Duration and number of claims
top_10_centers = df.groupby('Service Centre').agg(
    Mean_Payment_Duration=('Payment_Duration', 'mean'),
    Number_of_Claims=('Payment_Duration', 'count')
).reset_index()

# Sort and get the top 10
top_10_centers = top_10_centers.sort_values(by='Mean_Payment_Duration', ascending=False).head(10)

# Export to Excel
top_10_centers.to_excel("Top10_ServiceCenters_By_PaymentDuration.xlsx", index=False)

# Display result
print("Top 10 Service Centers by Average Payment Duration")
print(top_10_centers)
```

```
Top 10 Service Centers by Average Payment Duration
                               Service Centre  Mean_Payment_Duration  \
77                             CRESCENT CLINIC              45.000000
80                                  DAY CLINIC              43.000000
40   BUSAMED BRAM FISCHER INT AIRPORT HOSPITAL              42.000000
541                             UNION HOSPITAL              41.666667
28                                  BLOCK ROOM              41.000000
476                           ROOMS COSMO CITY              39.000000
492    ROOMS RONDEBOSCH MEDICAL CENTRE PRIVATE              37.000000
499                      ROUXCOR HOUSE ROOMS              36.000000
107               EMALAHLENI PRIVATE HOSPITAL              36.000000
297            MEDICLINIC MUELMED SUITES ROOMS              35.000000

     Number_of_Claims
77                  1
80                  1
40                  2
541                 3
28                  1
476                 1
492                 1
499                 1
107                 1
297                 1
```

```
# Group by Specialty and calculate mean Payment Duration and number of claims
top_10_specialties = df.groupby('Specialty').agg(
    Mean_Payment_Duration=('Payment_Duration', 'mean'),
    Number_of_Claims=('Payment_Duration', 'count')
).reset_index()

# Sort and get the top 10
top_10_specialties = top_10_specialties.sort_values(by='Mean_Payment_Duration', ascending=False).head(10)

# Export to Excel
top_10_specialties.to_excel("Top10_Specialties_By_PaymentDuration.xlsx", index=False)

# Display result
print("Top 10 Specialties by Average Payment Duration")
print(top_10_specialties)
```

```
Top 10 Specialties by Average Payment Duration
                                        Specialty  Mean_Payment_Duration  \
```

```
49             (086 003) Educational Psychologist             36.000000
35   (042) General Surgeon (003) Gastroenterology             20.181159
11            (018) Physician (006) Critical Care             19.666667
16                              (020) Neurologist             18.990741
76                                     Dietitian             18.000000
104               SAOPA (Orthotist & Prosthetist)             17.666667
28       (036) Plastic and Reconstructive Surgeon             16.411765
92                          Otorhinolaryngologist             16.000000
26          (032) Paediatrician (009) Pulmonology             15.904762
30                    (038) Diagnostic Radiologist             15.884615

     Number_of_Claims
49                  1
35                138
11                105
16                108
76                  8
104                 3
28                 34
92                 18
26                 21
30                 26
```

```
# Define target and drop from features
y = df['Payment_Duration']
X = df.drop(columns=['Payment_Duration'])

# One-hot encode categorical columns
categorical_cols = X.select_dtypes(include=['category', 'object']).columns
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=50)
print("Train shape:", X_train.shape, "Test shape:", X_test.shape)

# Initialize regression models
models = {
    'Linear Regression': LinearRegression(),
    'Decision Tree': DecisionTreeRegressor(random_state=50),
    'Gradient Boosting': GradientBoostingRegressor(random_state=50),
    #'Neural Network': MLPRegressor(max_iter=1000, random_state=50),
    'Lasso Regression': LassoCV(cv=5, random_state=50, max_iter=10000),
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state=50),

}


# Feature selection (top 50)
selector = SelectKBest(score_func=f_regression, k=50)
X_selected = selector.fit_transform(X, y)
selected_columns = X.columns[selector.get_support()]
print("Top selected features:")
print(selected_columns)

# Train, predict, and evaluate each model
for name, model in models.items():
    print(f"\nTraining and evaluating model: {name}")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    print(f"Mean Squared Error (MSE): {mse:.2f}")
    print(f"Mean Absolute Error (MAE): {mae:.2f}")
    print(f"R² Score: {r2:.4f}")
```

```
        'Service Centre_NETCARE ST AUGUSTINES HOSPITAL',
```

```
'ICD10 Description_Injury of spleen, without open wound into cavity'
```

```
       'ICD10 Description_Injury of spleen, without open wound into cavity',
       'ICD10 Description_Malignant neoplasm, thoracic part of oesophagus',
       'ICD10 Description_Mixed cortical and subcortical vascular dementia',
       'ICD10 Description_Open wound of other and unspecified parts of abdomen',
       'ICD10 Description_Open wound of scalp',
       'ICD10 Description_Other iridocyclitis',
       'ICD10 Description_Parkinson's disease',
       'ICD10 Description_Respiratory distress syndrome of newborn',
       'ICD10 Description_Stroke, not specified as haemorrhage or infarction',
       'ICD10 Description_Unknown',
       'ICD10 Description_Vascular headache, not elsewhere classified',
       'Gender_Unknown', 'Specialty_(018) Physician',
       'Specialty_(018) Physician (006) Critical Care',
       'Specialty_(020) Neurologist', 'Specialty_(028) Orthopaedic Surgeon',
       'Specialty_(032) Paediatrician',
       'Specialty_(042) General Surgeon (003) Gastroenterology',
       'Specialty_Physician', 'POSSIBLE PMB CALCULATED_PMB',
       'CONTRACTED/NON-CONTRACTED_NON-CONTRACTED'],
      dtype='object')

Training and evaluating model: Linear Regression
Mean Squared Error (MSE): 77.32
Mean Absolute Error (MAE): 5.74
R² Score: 0.0448

Training and evaluating model: Decision Tree
Mean Squared Error (MSE): 93.64
Mean Absolute Error (MAE): 5.78
R² Score: -0.1567

Training and evaluating model: Gradient Boosting
Mean Squared Error (MSE): 67.43
Mean Absolute Error (MAE): 5.48
R² Score: 0.1670

Training and evaluating model: Lasso Regression
Mean Squared Error (MSE): 80.10
Mean Absolute Error (MAE): 6.08
R² Score: 0.0105

Training and evaluating model: Random Forest
Mean Squared Error (MSE): 65.66
Mean Absolute Error (MAE): 4.96
```

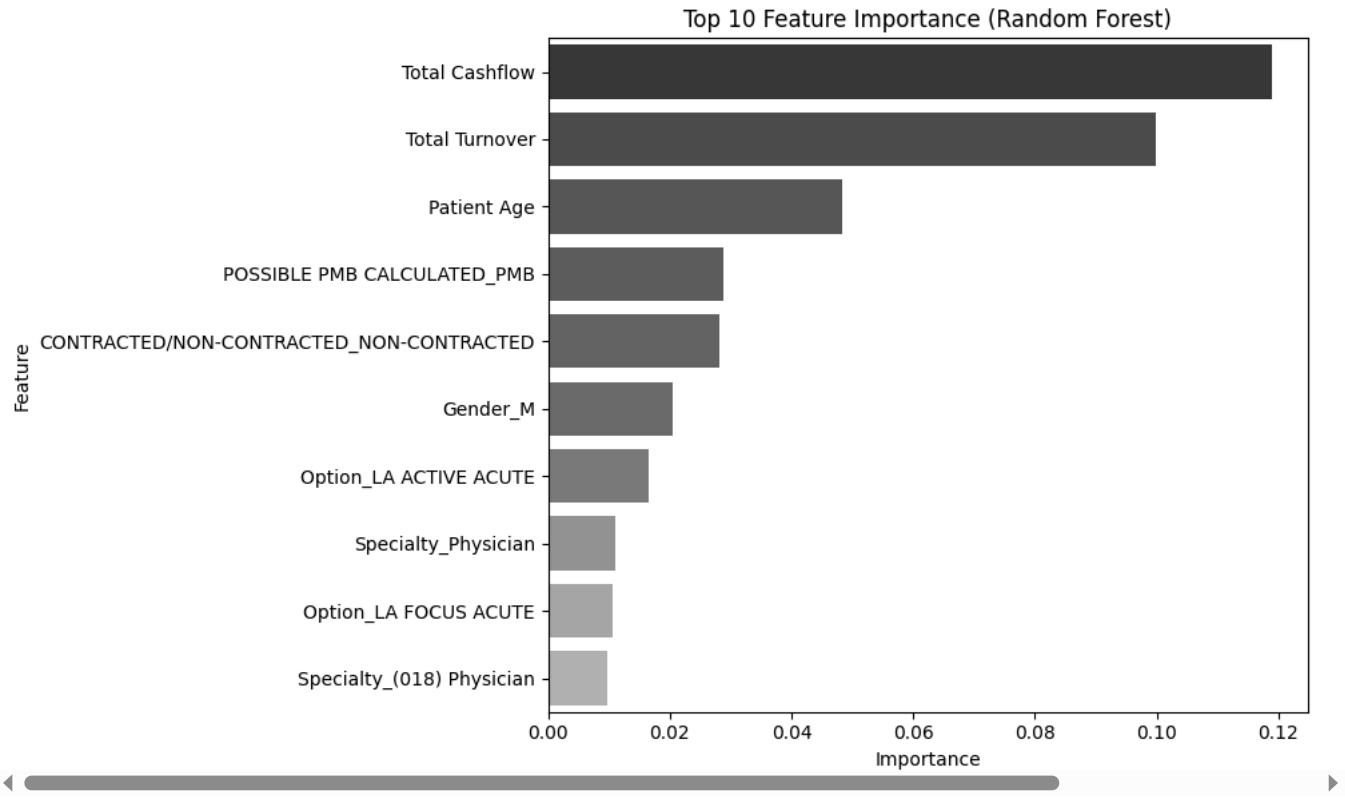
```
# --- Feature Importance from Random Forest ---
rf_model = models['Random Forest']
rf_importances = rf_model.feature_importances_
feature_importances = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': rf_importances
}).sort_values(by='Importance', ascending=False).head(10)

# --- Plot Top 10 Feature Importances ---
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=feature_importances, palette='viridis')
plt.title('Top 10 Feature Importance (Random Forest)')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.tight_layout()
plt.savefig('feature_importance.png')
plt.show()
```

```
<ipython-input-85-9e59db7caadf>:11: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `le

  sns.barplot(x='Importance', y='Feature', data=feature_importances, palette='viridis')
```




```
from sklearn.feature_selection import SelectKBest, f_regression
import pandas as pd

# === 1. Select top 100 features using SelectKBest on full dataset ===
selector_full = SelectKBest(score_func=f_regression, k=100)
X_selected_full = selector_full.fit_transform(X, y)

# === 2. Fit the best-performing model on the full dataset ===
best_model = RandomForestRegressor(n_estimators=100, random_state=50)
best_model.fit(X_selected_full, y)

# === 3. Predict Payment Duration for the entire dataset ===
y_pred_full = best_model.predict(X_selected_full)

# === 4. Reload the original (unprocessed) dataset ===
excel_file = 'LA_Health_V2.xlsx'  # or the correct file name/path
original_data_full = pd.read_excel(excel_file).reset_index()  # Adds index for tracking

# === 5. Create DataFrame for predictions and merge ===
predictions_df = pd.DataFrame({
    'index': original_data_full['index'],
    'Predicted_Payment_Duration': y_pred_full
})

merged_data = original_data_full.merge(predictions_df, on='index', how='left')

# === 6. Drop index if not needed ===
merged_data.drop(columns=['index'], inplace=True)

# === 7. Save to Excel ===
output_file_path = '/content/drive/My Drive/Colab Notebooks/Data/Predictions_with_RandomForest.xlsx'
merged_data.to_excel(output_file_path, index=False)

print(f"☑ Full dataset with predictions saved to: {output_file_path}")
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
<ipython-input-92-dabed9f77af2> in <cell line: 0>()
     15 # === 4. Reload the original (unprocessed) dataset ===
     16 excel_file = 'LA_Health_V2.xlsx'  # or the correct file name/path
---> 17 original_data_full = pd.read_excel(excel_file).reset_index()  # Adds index for tracking
     18
     19 # === 5. Create DataFrame for predictions and merge ===
```

3 frames